



UNIVERSIDADE FEDERAL DO RIO GRANDE DO NORTE
INSTITUTO METRÓPOLE DIGITAL
CURSO DE BACHARELADO EM TECNOLOGIA DA INFORMAÇÃO

Relatório da Unidade 2

Pedro Lucas Barbosa Nascimento

Natal/RN, 2025

Introdução

Este relatório detalha a evolução do projeto "Sistema de Vendas Super Conveniência 24H" durante a Unidade 2. O foco principal desta fase foi aprimorar a resiliência e a escalabilidade do sistema, atendendo diretamente às melhorias que identifiquei na U1, como a falta de persistência de dados e a limitação de um vetor estático.

O sistema simula um Ponto de Venda (PDV) e controle de estoque. Se na U1 o foco foi consolidar a lógica de controle de fluxo (if/else, switch, for, while) sobre um estoque de tamanho fixo, a U2 foi dedicada a incorporar os conceitos de programação avançada em C para permitir que o estoque armazene e carregue o estado do estoque (produtos e quantidades) em um arquivo (estoque.dat), e seja mais dinâmico, gerenciando o array de produtos com Alocação Dinâmica (malloc, realloc), eliminando o limite fixo de capacidade.

Metodologia

Ferramentas Utilizadas: O ambiente de desenvolvimento permaneceu o mesmo: o código foi escrito no Visual Studio Code, compilado com o GCC no Windows 11.

Abordagem de Desenvolvimento: A principal abordagem metodológica foi a refatoração do vetor de produtos, migrando de uma variável global estática (Produto estoque[MAX_CAPACIDADE]) para um **ponteiro** (Produto *estoque) gerenciado dentro da função main.

Organização do Código

A modularidade do código (já implementada na U1) provou ser vital. Como a organização do código facilita a manutenção? Ao separar a lógica de I/O de arquivo em funções dedicadas (carregar_estoque, salvar_estoque), consegui isolar o código complexo de manipulação de ponteiros e leitura de strings. Isso permite que eu atualize o formato do arquivo ou a estratégia de realocação sem ter que tocar na lógica de venda, tornando o código mais limpo e fácil de debugar.

Análise

Quais conceitos da U2 foram aplicados no projeto? Todos os conceitos centrais foram aplicados: Ponteiros, Alocação Dinâmica, Strings e Estruturas de Repetição Aninhadas.

Strings: As strings (char nome[MAX_NOME]) são usadas para identificar os produtos.

Utilização no Projeto: Além de armazenar nomes, a manipulação de strings foi crucial para a persistência dos dados.

Funções de Manipulação Implementadas: Utilizei fprintf para escrever os campos da struct Produto no arquivo, separando os valores com um delimitador (;). Ex: fprintf(arquivo, "%d;%s;%.2f;%d\n", ...). Utilizei também fscanf com um especificador de formato que lida com strings delimitadas: %49[^;]. Isto garante que a string seja lida até o próximo delimitador (;).

Repetições Aninhadas: A principal repetição aninhada está no main (do-while principal do menu) com o loop de Carrinho de Compras (while (continuar == 1)) dentro do case 2 (Iniciar Venda). Sobre os casos de uso, O loop interno permite que o vendedor escaneie/digite múltiplos IDs de produtos (repetição) até finalizar a transação, sem ter que reiniciar o sistema de venda a cada item.

Matrizes: A estrutura complexa foi implementada como um Vetor Dinâmico de Structs. Em C, isso é um array unidimensional de estruturas, onde cada índice é um objeto Produto completo, funcionando como uma "linha" de dados.

Implementação: Produto *estoque

Operações Suportadas: Leitura, busca sequencial, e a principal: **expansão** (via realloc) e **persistência** (via I/O de arquivo).

Estratégias de Percorrimento: Utilizamos o laço for com notação de array (estoque[i]) nas funções listar_estoque, buscar_indice_por_id e salvar_estoque.

Ponteiros

Como os ponteiros foram utilizados para melhorar a eficiência?

Alocação Dinâmica: Permitem gerenciar grandes blocos de memória apenas com o endereço inicial (Produto *estoque), o que é muito mais eficiente do que copiar todo o array em cada modificação. Na função realizar_desconto, a passagem do subtotal através de um ponteiro

(float *subtotal) evita a necessidade de return de múltiplos valores ou de usar variáveis globais, o que é mais limpo e eficiente.

Alocação Dinâmica

Quais vantagens a alocação dinâmica trouxe ao projeto?

A principal vantagem foi a **escalabilidade**. O projeto não está mais limitado a uma constante CAPACIDADE_MAXIMA pré-definida.

Onde foi necessária? Na função `carregar_estoque` e, conceitualmente, na função de adicionar novos produtos (mesmo que esta versão tenha focado na persistência).

Como é gerenciada a memória? A memória é gerenciada por meio de três chamadas principais: `malloc`, `realloc` e `free`.

Qual a estratégia para gerenciamento de memória?

Minha estratégia é o gerenciamento de ciclo de vida centralizado. O ponteiro `estoque` é declarado no `main`, e as funções de I/O apenas o manipulam, mas não o liberam. O código verifica se `malloc` ou `realloc` retornaram `NULL`. Se a alocação falhar, uma mensagem de erro é exibida e o programa tenta continuar com o que tem (ou encerra, se a falha for inicial), prevenindo *crashes*. A garantia é dada por um único e crucial ponto: a chamada `free(estoque)` dentro do `case 0` antes do `main` retornar. Se esta linha for omitida, o bloco de memória alocado permaneceria reservado até que o programa fosse encerrado pelo sistema operacional, caracterizando um vazamento.

Quais foram os principais desafios técnicos enfrentados?

O principal desafio foi a implementação correta de `realloc` e a garantia de que o novo endereço de memória fosse refletido corretamente no `main`.

Dificuldades com Ponteiros e Gerenciamento de Memória

Tive problemas na função de carregamento quando o array precisava crescer. Se eu tentasse apenas:

```
// Tentativa inicial ERRADA dentro da função
estoque = realloc(estoque, nova_capacidade * sizeof(Produto));
// O novo endereço é atribuído à cópia local do ponteiro 'estoque'
```

O ponteiro no main continuava apontando para o endereço antigo e incorreto, o que levaria a erros de segmentação.

A solução foi garantir que as variáveis de controle (num_produtos e capacidade_atual) fossem passadas por ponteiro (int *) para que a função carregar_estoque pudesse alterar seus valores no escopo do main. Similarmente, a função (se fosse uma de adição) precisaria retornar o novo endereço do array ou receber o ponteiro por referência dupla. No caso de carregar_estoque, a solução foi retornar o novo ponteiro alocado (return estoque;), atribuindo-o diretamente no main (estoque = carregar_estoque(...)).

Aprendizados Importantes

A persistência exige mais do que apenas abrir um arquivo; exige um formato de dados rigoroso (como usar o delimitador ;) e a manipulação cuidadosa das strings. A responsabilidade de liberar a memória é do desenvolvedor. A inclusão da chamada free(estoque) é o que transforma o código funcional em código seguro.

Conclusão

Síntese dos Aprendizados

A Unidade 2 foi um salto qualitativo. Consegui transformar o Sistema de Vendas em uma aplicação que simula o uso real, mantendo os dados salvos e permitindo que o estoque cresça infinitamente. Os ponteiros e a alocação dinâmica não são apenas conceitos teóricos, mas sim ferramentas práticas para construir softwares escaláveis.

Na U1, eu questionei como cadastrar novas mercadorias e como salvar o estoque. Hoje, o projeto responde a essas questões com I/O de arquivo e alocação dinâmica. O código da U2 é mais complexo, mas muito mais poderoso, pois resolve as limitações estruturais da versão anterior. Para mim, o projeto já está bem completo, mas eu poderia implementar melhorias futuras, como otimizar sua performance e também introduzir uma interface mais interativa para o usuário.